# Chapter 7

Importing Data

by Lorraine Gaudio

Version May 29, 2026

## Contents



# 1 Overview

In earlier chapters, you learned how to write R code, use built-in functions, and work with missing values. In this chapter you do something more realistic: you bring datasets into R the way researchers do in actual projects.

Most import problems are not "R problems." They are **file management problems**: the file is in the wrong folder, the file name doesn't match, or the package you need wasn't loaded. You will learn a simple, reliable workflow: download the data, put it in the same folder as your R Notebook, confirm R can see it, and then import it with the correct function.

Watch how this chapter models autonomy. Build these habits, documenting them in your practice space report. Autonomy means taking initiative when something does not work: checking where R is looking, confirming the file name, reading the message, and reaching out for support when you are genuinely stuck. The goal is not to solve every problem alone. The goal is to stay in control of your learning instead of letting a file-path error stop your progress.

This chapter is also structured differently: Many of the Practice Space tasks are built into the guided activity. If you complete each step as you go, you will finish the chapter with a working notebook that can Knit successfully and reload your datasets from a clean start.

By the end of Chapter 7, you will be able to:

- Define tabular data and describe how rows and columns map to observations and variables in a data frame.
- Explain how the working directory affects file imports and compare the working directory used in the Console vs. during Knit.
- Organize downloaded data files in the correct folder and verify file visibility using `list.files()`.
- Install required packages once and load required packages in an R Notebook so code runs from top to bottom when knitted.
- Load datasets from Base R and from installed packages using `data()` and confirm that the dataset is available in the Environment.
- Import CSV and Excel files using `read_csv()` and `read_excel()` and interpret the import feedback (rows/columns and column types at a basic level).
- Troubleshoot common import failures (missing packages, wrong file location, wrong file name) using targeted checks (`getwd()`, `list.files()`, `library()`).

# 2 Data Management

One of the most common stumbling blocks to importing data is knowing where a file is stored on your computer.

A **file path** is the address to a file. When you write import code, R uses the file path to find the file you want to load. If R looks in the wrong folder, the import will fail even if your code is almost correct.

The first step to loading data into R is learning how to manage your files and folders so that R can find your data when you run code in the Console, run code in an Rmd file, or knit a Notebook, R Markdown, or Quarto document.

## 2.1 Folder Structure

Your course work should be stored in your main course folder:

`Intro_to_R`

For this chapter, create a new folder inside your `Intro_to_R` folder called:

`Module_4`

**Tip:** Use `Module_4` instead of `Module 4` because spaces in folder or file names can cause issues in RStudio. R's file-handling functions and the underlying operating system (especially Windows) have specific rules for

how paths are interpreted. For example, when you try to open an R Markdown file with spaces in its name, the IDE may treat the space as a special character and fail to locate it.

Your folder structure should look like this:

```
Intro_to_R/
  Module_4/
```

### 2.1.1 How to read folder structure diagrams

Here is how to read that folder structure:

- `Intro_to_R/` is your main course folder.
- `Module_4/` means `Module_4` is inside the `Intro_to_R` folder.
- The `/` at the end means the item is a folder, not a file.
- The indentation matters. Anything indented farther to the right is nested inside the folder above it.

You may also see folder diagrams that use more symbols:

```
Intro_to_R/
  Module_1/
      rstudio.png
      chapter1_practice.RData
      chapter2_practice.R
      chapter2_practice.RData
      chapter2_notes.R
  Module_2/
      chapter3_practice.Rmd
      chapter3_practice.RData
      chapter3_notes.Rmd
      chapter4_practice.Rmd
      chapter4_notes.Rmd
  Module_3/
      chapter5_practice.Rmd
      chapter5_practice.html
      chapter5_notes.Rmd
      chapter6_practice.Rmd
      chapter6_practice.html
      chapter6_notes.Rmd
  Module_4/
      chapter7_practice.Rmd
      chapter7_practice.html
      Values_Transport_dplace.csv
      car_prices.xlsx
```

Here is how to read this diagram:

- Intro_to_R/ is the main course folder.
- `Module_1/`, `Module_2/`, `Module_3/`, and `Module_4/` are folders inside `Intro_to_R/.`
- The `/` at the end means the item is a folder, not a file.
- ├── marks an item when more files or folders come after it at the same level.
- └── marks the last item inside a folder.
- │ is a visual guide. It shows that the folder structure continues below at that same level.
- Indentation shows nesting. Files indented under `Module_4/` are inside the `Module_4/` folder.
- `chapter7_practice.Rmd`, `Values_Transport_dplace.csv`, and `car_prices.xlsx` are inside Module_4/.
- Those three files are not directly inside `Intro_to_R/`; they are one level deeper.

Organize your course work in a folder structure that makes sense to you. The important thing is that your **Rmd file and your data files are in the same folder** so that R can find them when you run code or knit your notebook.

---

This activity will walk you through the process of placing the activity Rmd file and data files in the same folder. We'll start by creating and saving the R Notebook Rmd file inside your new `Module_4` folder.

## 2.2 Create/Save Chapter 7 Notebook

Create an R Notebook

1. Open an R Notebook: Go to `File > New File > R Notebook`
2. Edit the YAML Header and delete the default text. The YAML header should look like this:

```
---
title: "Chapter 7 Practice"
author: "[Your Name]"
date: "`r Sys.Date()`"
output: html_notebook
---
```

3. Change `[Your Name]` to your actual name and clear the default text in the body of the notebook.
4. Save your R Notebook `File` > `Save As...`, choose your course folder, name the file `chapter7_practice`.

```
Intro_to_R/
  Module_4/
      chapter7_practice.Rmd
```

> **Notice:** The guided activity is using `chapter7_practice.Rmd` as the file name because completing this guided activity will help you get started in the Practice Space file submission.

## 2.3 Downloading Data

You will need two data files for this chapter:

- `Values_Transport_dplace.csv`
- `car_prices.xlsx`

Find these data files in the **Importing Data Guided Activity** instructions page. Click the download links. Do not open the downloaded files.

> Common Pain Point: After you download `Values_Transport_dplace.csv`, do **not** open it. This is especially important on some **Mac** computers. When you double-click a `.csv` file, your Mac may open it in **Numbers**. A `.numbers` file is a completely different file format used by Apple Numbers.

After you download the files, move them out of your `Downloads/` folder and into your `Module_4/` folder.

Moving the files is part of the data workflow. R can only import a file if the file is in the place where R is looking or if you give R the correct file path. In this chapter, we are keeping the workflow simple by putting the notebook and data files in the same folder.

Download the data files for this chapter and place them in your `Module_4/` folder with your Rmd file. **This is Task 1 of the Practice Space at the end of this chapter.**

By the time you are done, your folder structure should look like this:

```
Intro_to_R/
  Module_4/
      chapter7_practice.Rmd
      Values_Transport_dplace.csv
      car_prices.xlsx
```

Your Rmd file and both data files should be in the same folder.

---

### 2.3.1 Need help moving the downloaded files?

If you already know how to move files from Downloads into your `Module_4` folder, skip ahead to Working Directory.

If you are not sure how to move downloaded files, follow the steps for your computer: macOS or Windows.

#### 2.3.1.1 macOS Move files from Downloads into `Module_4`

1. Open **Finder**.
   - Finder is the blue-and-white face icon in the Dock.
2. Open your **Downloads** folder.
   - In the Finder sidebar, click **Downloads**.
   - Look for these two files:
     - `Values_Transport_dplace.csv`
     - `car_prices.xlsx`
3. Open a second Finder window.
   - At the top of your screen, click **File → New Finder Window**.
   - You can also press **Command + N**.
   - If this does not work, click once on Finder first, then try again.
4. In the second Finder window, go to your course folder.
   - Find and open your `Intro_to_R` folder.
   - Inside `Intro_to_R`, open the `Module_4` folder.
5. Put the two Finder windows next to each other.
   - One window should show **Downloads**.
   - One window should show `Intro_to_R/Module_4/`.
6. Select both data files in Downloads.
   - Click `Values_Transport_dplace.csv`.
   - Hold the **Command** key.
   - While holding **Command**, click `car_prices.xlsx`.
   - Both files should now be selected.
7. Drag the selected files into the `Module_4` Finder window.
   - Click and hold on one of the selected files.
   - Drag the files into the open `Module_4` window.
   - Release the mouse or trackpad.

Verify it worked. - Click inside the `Module_4` Finder window. - You should see: - `chapter7_practice.Rmd` - `Values_Transport_dplace.csv` - `car_prices.xlsx`

If you can see all three files in the `Module_4` window, the move was successful.

If something is wrong:

- If one or both data files are still in Downloads, drag them into `Module_4` again.
- If you cannot find a file, return to Canvas and download it again.
- If dragging does not work, try:
  - selecting the file and pressing **Command + C** to copy,

  - opening the `Module_4` folder,
  - then pressing **Command + V** to paste.
- If the file opens in Numbers or Excel instead of moving, close the program without saving changes and return to Finder.
- If you accidentally changed the file type or are unsure what happened, delete the file and download a fresh copy from Canvas.

R will not be able to import the files correctly unless the notebook file and both data files are together inside `Intro_to_R/Module_4/`.

Without the files in the correct location, you cannot progress forward with the activity. Continue for a few minutes troubleshooting but if you feel stuck, reach out for help from a classmate, friend, your instructor, or visit The Zone

---

If your files are in the correct folder, move to Working Directory

#### 2.3.1.2 Windows Move files from Downloads into `Module_4`

1. Open **File Explorer**.
   - File Explorer is the yellow folder icon on the taskbar.
   - You can also right-click the Start button and choose **File Explorer**.
2. Open your **Downloads** folder.
   - In the left sidebar, click **Downloads**.
   - Look for these two files:
     - `Values_Transport_dplace.csv`
     - `car_prices.xlsx`
3. Open a second File Explorer window.
   - Right-click the File Explorer icon on the taskbar.
   - Click **File Explorer** to open another window.
4. In the second File Explorer window, go to your course folder.
   - Find and open your `Intro_to_R` folder.
   - Inside `Intro_to_R`, open the `Module_4` folder.
5. Put the two File Explorer windows next to each other.
   - One window should show **Downloads**.
   - One window should show `Intro_to_R\Module_4\`.
6. Select both data files in Downloads.
   - Click `Values_Transport_dplace.csv`.
   - Hold the **Ctrl** key.
   - While holding **Ctrl**, click `car_prices.xlsx`.
   - Both files should now be selected.
7. Drag the selected files into the `Module_4` File Explorer window.
   - Click and hold on one of the selected files.
   - Drag the files into the open `Module_4` window.
   - Release the mouse.

Verify it worked. - Click inside the `Module_4` File Explorer window. - You should see: - `chapter7_practice.Rmd` - `Values_Transport_dplace.csv` - `car_prices.xlsx`

If you can see all three files in the `Module_4` window, the move was successful.

If something is wrong:

- If one or both data files are still in Downloads, drag them into `Module_4` again.
- If you cannot find a file, return to Canvas and download it again.
- If dragging does not work, try:
  - selecting the file and pressing **Ctrl + C** to copy,

  - opening the `Module_4` folder,
  - then pressing **Ctrl + V** to paste.
- If the file opens in Excel instead of moving, close Excel without saving changes and return to File Explorer.
- If you accidentally changed the file type or are unsure what happened, delete the file and download a fresh copy from Canvas.

R will not be able to import the files correctly unless the notebook file and both data files are together inside `Intro_to_R\Module_4\`.

Without the files in the correct location, you cannot progress forward with the activity. Continue for a few minutes troubleshooting but if you feel stuck, reach out for help from a classmate, friend, your instructor, or visit The Zone

---

If you can see the files in the correct location, then you are ready to move on to the next step: checking your **working directory** and confirming that R can see the files you just moved.

## 2.4 Working Directory

In Chapter 1, you learned how to use `getwd()` and `setwd()` to manage your working directory. This is crucial for importing data because R needs to know where to find the files you want to load.

**Running chunks interactively often uses the same session as the Console.** The real difference shows up when you Knit, because knitting runs in a fresh session and uses the Notebook file location as the working directory. That means your notebook must include everything needed to run from the top, including correct file paths.

Make R able to find your data files when you (1) run code in the Console and (2) when you Knit an R Notebook document.

### 2.4.1 Console

**Check your working directory in the Console**

Run in the **Console**.

```
#  Type this in the Console
getwd()
list.files()
```

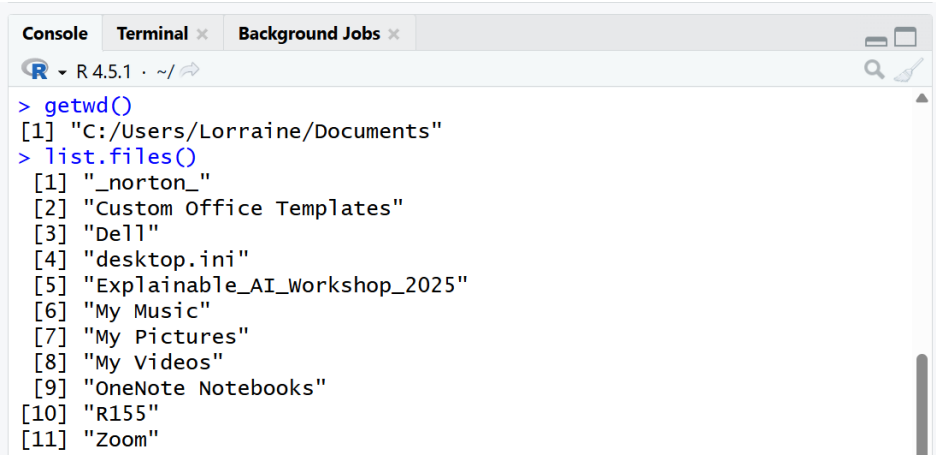


Figure 1: Image of the instructor's Console after running getwd() and list.files() as an example of an output.

In the knitted notebook output, look directly under the `getwd()` and `list.files()` in the console output. Confirm that the folder path matches where you saved `chapter7_practice.Rmd`. Then check whether the file list includes the notebook file itself.

When you run `getwd()`, you are checking where R is currently "looking." When you run `list.files()`, you are checking what R can actually see from that location.

If the file you need is not listed, your next move is to check whether the file is in the right folder or whether the working directory is set to the folder location where the desired file is saved.

---

The Notebook works differently because when you Knit, R starts a fresh session and sets the working directory to the folder where the Notebook is saved. Let's see that in action!

### 2.4.2 R Notebook

**Check your working directory in the Notebook**

### 2.4.3 Step 1

Add a new code chunk with the same code as above.

```
#  Type this in your notebook
getwd()
list.files()
```

### 2.4.4 Step 2

**Test knit.**

Click the Knit button at the top of the R Notebook editor pane. This will run all the code in your notebook from top to bottom in a fresh R session and create an HTML document as output.

### 2.4.5 Step 3

**Verify**

Review the output under the `getwd()` and `list.files()` chunk in the resulting HTML.

In the knitted notebook output, look directly under the `getwd()` and `list.files()` chunk. Confirm that the folder path matches where you saved `chapter7_practice.Rmd`. Then check whether the file list includes the notebook file itself.

You should see the files for this chapter listed in the output:

```
[1] "car_prices.xlsx"              "chapter7_practice.Rmd"        "Values_Transport_dplace.csv"
```

The working directory of an R Notebook is the folder where the notebook is saved.

**Write a memo** in your Notebook explaining the difference in file paths when you run the code in the Console vs. the Knit R Notebook. **This is Task 2 of the Practice Space at the end of this chapter.**

What this means for loading data is that R must be able to find your data file from the working directory it is using. If you are working in the Console or an R script, check or set the working directory yourself. When you are working in an Rmd (or qmd) file, you will keep the data files in the same folder as the Rmd file unless you provide a specific file path to use.

> In `Chapter 9: Git and GitHub`, you will learn how to organize data files in a subfolder.

---

For the remainder of this chapter, we will use the workflow where you load data files that are in the same folder as your R Notebook. This is a common and simple way to manage your files.

# 3 Import in R

This section shows you how to prepare R to import datasets. You will (1) install the packages needed for this chapter one time, and (2) load those packages every time you start a new R session so your notebook can run from top to bottom when you Knit.

## 3.1 Packages refresher

To import data, we will need to use functions from additional packages that are not part of the base installation of R. A **package** is a collection of functions, data, and documentation that extends the capabilities of R. You were first introduced to the concept of packages in chapter 3 when you downloaded packages (e.g., `rmarkdown`) to run your R Notebook.

### 3.1.1 Installing Packages

You need to download the package before you can use it. **You only need to install the package once.** Packages will need to be updated from time to time.

Install the packages you need for this chapter as a *one time action*.

Method A: **GUI method to install packages.**

**Bottom-right pane → Packages tab → Install → type the package name.**

In the bottom-right **Files Pane**, you will see a tab called "Packages." This is where RStudio lists the packages you have installed. You can install packages by clicking the Install button.

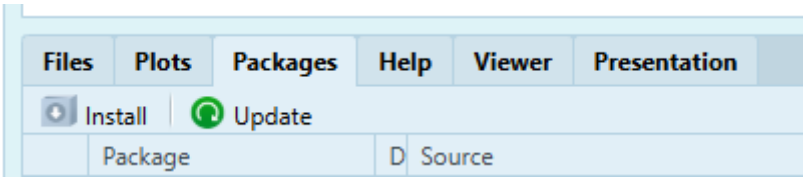


Figure 2: Click the Install button from the Packages tab then type `dslabs` in the Packages box and click Install to download dslabs from the Package Repository CRAN

After you click Install, notice the lines of code and message(s) that result in the Console. The code is what you would have to type if you were installing packages without using the GUI (Method B).

Method B: **Type the code in the Console to install packages.**

Often times, you will use the function `install.packages()` to install a package. In this chapter, we need the packages `readr`, `readxl`, `dslabs` package, which provides functions for reading and writing data files.

Install the packages `readr`, `readxl`, and `dslabs` using `install.packages()`.

```
#  Type this in your Console to install packages
install.packages("dslabs")                # You can install one package at a time
install.packages(c("readr", "readxl"))    # Or you use c()
```

The `install.packages()` function is used to install packages from CRAN. It is normal to see a Warning message in your console. This code installs the `dslabs`, `readr`, and `readxl` packages. The `c()` function combines the package names into a vector, allowing you to install multiple packages at once.

**Test knit.**

Common Pain Point: `install.packages()` lines will not run successfully in a knitted notebook because of the way that knitting works. If you include `install.packages()` lines in your R Notebook, make sure to comment them out with `#` after you run them once in the Console.

```
#  Check
packageVersion("dslabs")
```

```
## [1] '0.9.1'
```

```
packageVersion("readr")
```

```
## [1] '2.2.0'
```

```
packageVersion("readxl")
```

```
## [1] '1.5.0'
```

Each `packageVersion()` line should print a version number. A version number means the package is installed on your computer. If you get an error saying there is no package with that name, the package did not install successfully.

---

After installing packages, you need to load them into your R session to use their functions. This is done with the `library()` function, which we will cover next.

### 3.1.2 Loading Packages

After installing the package, you need to load it into your R session using the `library()` function. This makes the functions in the package available for use. **Every time you start a new R session, you will need to load the packages you want to use.** To ensure your R Notebook works when you Knit it, you should include the `library()` calls in your R Notebook.

**Tip:** Add headings and format your report as you work. Add the heading # Library Packages.

Load the packages `readr`, `readxl`, and `dslabs` into your R session **using a notebook code chunk** so it load *every time* when you Knit. **This is Task 3 of the Practice Space at the end of this chapter.**

```
#  Type this in a code chunk in the R Notebook to load packages
library(dslabs)  # For more datasets
```

```
## Warning: package 'dslabs' was built under R version 4.5.3
```

```
library(readr)   # For reading CSV files
```

```
## Warning: package 'readr' was built under R version 4.5.3
```

```
library(readxl)  # For reading Excel files
```

```
## Warning: package 'readxl' was built under R version 4.5.3
```

`library()` may print startup messages, and those messages are not automatically a problem. An error means the package did not load. A normal message usually tells you additional information such as the version.

**Test knit.**

---

Your folder system, notebook, and packages are now set up to import data. The next step is to learn how to load different sources of tabular data.

# 4 Tabular data

**Tabular data** is data that is arranged in a table format where rows represent observations and columns represent variables. This is the most common format for datasets and what most people think of when they think about data. When you import data into R, it is often stored as a data frame.

A **data frame** (df) is the most common way to store tabular data in R. It is a two-dimensional table made of rows and columns. Data frames are rectangular. Every column must have the same number of rows. In other words, each column must be the same length, because every row must have a value (or an `NA`) for every column.

**Rows** usually represent observations (cases, people, states, experiments, etc.). **Columns** usually represent variables (measurements you recorded). A data frame can store different data types in different columns (numbers, characters, logical values, etc.). A practical way to think about a data frame is a collection of columns, where each column is a vector.

---

This section shows two common ways to bring data into your R session.

1. You can load datasets that already come with R or with an installed package. These are fast and reliable for practice.
2. You can import data files from your computer, such as CSV or Excel files. These require the correct package, file name, and file path.

We will practice both in this chapter because future chapters will ask you to use built-in package data and imported data files.

## 4.1 Package Datasets

To load package datasets, use the `data()` function with the name of the dataset and the package it comes from.

The basic SYNTAX: `data("dataset_name", package = "package_name")`

### 4.1.1 Base R Datasets

Base R comes with a set of built-in `datasets` that you can use for practice and learning. These datasets are part of the base R installation and are available without needing to install any additional packages.

View a list of datasets that are already in R

```
#  Type this in your Console to see the available datasets in base R
data(package = "datasets")
```

This opens a list of datasets that come from R by default. You can see the name of the dataset and a brief description of it.

These datasets are already available but you can make their availability more explicit. You'll do this by loading the selected dataset into your environment using the `data()` function. Throughout the remainder of this course, we will often use `mtcars` from base R.

Load the `mtcars` dataset from base R. **This is Task 4 of the Practice Space at the end of this chapter.**

```
#  Type this in your R Notebook to load the mtcars dataset
data("mtcars")
```

After running this code, the `mtcars` dataset is loaded into your R environment.

Look in the Environment pane for `mtcars`. You can click the object in your environment to view the data. Alternatively, you can run the code `View(mtcars)`. Either method will open a new window in RStudio that displays the data.

> **Notice:** Something interesting about the `mtcars` dataset is that it has row names (labels for rows). These are not a normal "data column." The metadata attached to the table and row names can be convenient for display, but they can also cause confusion later because they don't behave like a regular variable. In most real projects, identifiers should be stored as a real column, not as row names.

**Test knit.**

### 4.1.2 Curated Datasets

In addition to the base R datasets, there are packages that you can install that provide additional datasets for practice and learning. These datasets are often curated and come with documentation that explains the context and variables in the dataset. One popular package for curated datasets is `dslabs`, which provides a variety of datasets for data science education from Harvard/X data-science labs.

```
#  Type this in your Console to see what datasets are available from dslabs
data(package = "dslabs")
```

Load the `movielens` dataset from the `dslabs` package. **This is Task 5 of the Practice Space at the end of this chapter.**

```
#  Type this in your R Notebook to load the "movielens" dataset
data("movielens", package = "dslabs")
```

The `movielens` dataset from the `dslabs` package is loaded into the R environment.

You can click the object in your environment to view the data.

> Chapter 8 will teach you how to explore tabular data in more detail.

If a dataset does not appear, your next step is to check the sequence of your notebook. Code that loads packages and data must come before code that uses those objects.

**Test knit.**

---

These datasets are great for practice, but in real projects, you will often need to import data from external files. The next section shows you how to do that.

## 4.2 External Files

Often, you will need to load data from external files that you have downloaded or created. The most common file formats for data are **CSV** (`.csv`) and **Excel** (`.xlsx`). To load these files into R, you will need to use functions from the `readr` and `readxl` packages, respectively.

### 4.2.1 Read CSV

Importing a CSV (`read_csv`) requires the package `readr` that we loaded earlier. If you forget to load (`library()`) the `readr` package first, you will get an error message when you try to use the `read_csv()` function.

Because we are using an R Notebook, you will use a simple file path because your file is saved in the same location as the R Notebook.

The basic SYNTAX:

```
df <- read_csv("file.csv")
```

Import the CSV file `Values_Transport_dplace.csv` into R using `read_csv()` and store it as `dplace_df`. **This is Task 6 of the Practice Space at the end of this chapter.**

```
#  Type this in your R Notebook to import the CSV file
dplace_df <- read_csv("Values_Transport_dplace.csv")
```

```
## Rows: 185 Columns: 7
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (3): id, name, sub_case
## dbl (4): domainelement_pk, pk, valueset_pk, year
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

The `read_csv()` function reads the CSV file and creates `dplace_df`, a data frame that contains the data from the CSV file.

You can click on `dplace_df` in your Environment to view the imported data.

> **Notice:** When loading, R prints an import summary to confirm what it found. The `dplace_df` data frame contains 185 rows and 7 columns. `Delimiter: ","` means R detected a comma-separated file. The column specification shows how `readr` interpreted each column. In this case, `chr (3)` means 3 character columns: `id`, `name`, and `sub_case`. `dbl (4)` means 4 numeric columns: `domainelement_pk`, `pk`, `valueset_pk`, and `year`.

**Test knit.**

---

If the import does not work, read the first line of the error message and check the most likely problem first.

- If the error says the file cannot be opened or does not exist, R cannot find `Values_Transport_dplace.csv`.
  - Check the spelling, capitalization, underscores, and `.csv` ending.
  - Check that the file is in the same folder as your Rmd file.
- If the columns look wrong, the file may not have been read as a comma-separated file.
  - Make sure you did not open and resave the CSV file in Numbers or Excel.
  - If you are unsure, delete the file and download a fresh copy.
- If the error says could not find function `read_csv`, the `readr` package is not loaded.
  - Run `library(readr)` in a code chunk before the import code.

**Break Things!** Test a common error on purpose. Close and reopen RStudio, then run this line **before** running `library(readr)`:

```
dplace_df <- read_csv("Values_Transport_dplace.csv")
```

You should see an error saying R cannot find the `read_csv()` function. This does not mean the CSV file is broken. It means the package that contains `read_csv()` has not been loaded yet.

**Tip:** Any time the you reopen RStudio, run all code chunks to load your packages and data into R Studio again.

---

### 4.2.2 Read Excel

Importing an Excel file requires the package `readxl` that we loaded earlier. If you did not load the `readxl` package, you will get an error message when you try to use `read_excel()`.

**Type `read_excel()` in your notebook**

Saving the data file in the exact same folder as your R Notebook allows you to use the file name without a path.

```
df <- read_excel("file.xlsx")
```

Import the Excel file `car_prices.xlsx` to R using `read_excel()` and store it as `car_prices_df`. **This is Task 7 of the Practice Space at the end of this chapter.**

```
#  Type this in your R Notebook to import the Excel file
car_prices_df <- read_excel("car_prices.xlsx")
```

> **Notice: Be patient when you load an Excel file.** R may take a few seconds to read an Excel file, especially if the file is large, has complex formatting, or if you are importing more than one sheet. Very large Excel files can take longer to import. The limit is usually the computer's available memory and processing power, not RStudio itself. However, opening a very large dataset in RStudio's data viewer can slow down the RStudio window a lot.

The `read_excel()` function reads the Excel file and creates a data frame called `car_prices_df` that contains the data from the Excel file. You can click on `car_prices_df` in your environment to view the imported data.

**Tip:** If the Excel import fails, check if the Excel file is open on your computer. Always close the Excel file before trying to import it into R.

Look in the Environment pane for `car_prices_df`. Click the object or run `head(car_prices_df)` to confirm that the rows and columns look like car price data.

View the data frame by clicking on `car_prices_df` in your Environment pane or by running `View(car_prices_df)`.

If the Excel import does not look right, check three things before asking for help: whether the file is listed in `list.files()`, whether the file name is spelled exactly as `car_prices.xlsx`, and whether the workbook has more than one sheet.

**Test knit.**

#### 4.2.2.1 Multiple Sheets

Excel files can contain multiple sheets. By default, `read_excel()` reads the first sheet. If your Excel file has multiple sheets, you can specify which sheet to read using the `sheet` argument. You can specify the sheet by name or by number.

```
# Optional Example
excel_sheets("car_prices.xlsx")
```

```
## [1] "car_prices"
```

```
car_prices_sheet1 <- read_excel("car_prices.xlsx", sheet = 1)
```

The `excel_sheets()` function lists the names of the sheets in the Excel file. The `read_excel()` function with the `sheet` argument reads the specified sheet from the Excel file.

`sheet = 1` reads the first sheet but you can also specify the sheet by name. For example, `sheet = "car_prices"`.

**Test knit.**

---

# 5 Summary

In this chapter, you learned how datasets enter R and why file management is a common reason imports fail. We started by defining tabular data as a table where rows represent observations and columns represent

variables. You also learned that imported tabular data is usually stored in a data frame, which is rectangular: every column has the same number of rows, and missing values should appear as `NA`.

You also learned how the working directory controls where R looks for files. You used `getwd()` to print the current working directory and `list.files()` to list the files R can see in that folder. You compared running code in the Console to knitting an Rmd file, where code chunks usually use the Rmd file's folder as the working directory. To prevent "cannot open file" errors, you practiced saving downloaded data files in the same folder as your R Notebook and checking that the filenames appear in `list.files()` before importing.

Finally, you practiced the core import workflows used throughout the course. You installed packages with `install.packages()`, verified installation with `packageVersion()`, and loaded packages each session with `library()`. You loaded built-in and package datasets with `data()`, then imported external files with `readr::read_csv()` for CSV files and `readxl::read_excel()` for Excel files. When something did not work, you used targeted checks for the working directory, file visibility, and loaded packages.

The larger learning goal is that you now have a repeatable import workflow: place the file, check where R is looking, confirm R can see the file, load the right package, import the data, and verify the result. This supports autonomy because you have a clear next step when something goes wrong. You do not have to guess, freeze, or wait for someone to notice you are stuck; you can investigate the problem, gather evidence, and ask for targeted help when needed.

# 6 Chapter Terms

**Columns**: The vertical fields in a data frame or table. In tidy tabular data, each column usually represents one variable, such as a name, category, measurement, count, or logical value. In R, each column in a data frame is a vector, and all columns must have the same number of rows.

**CSV** (Comma-Separated Values): A plain-text file (`.csv`) for rectangular (row-and-column) data where each row is a line and values are separated by commas. Because it's plain text, CSV is widely compatible across software, but it does not reliably preserve spreadsheet features like formulas, cell formatting, or multiple sheets.

**Data Frame**: A two-dimensional R object for tabular data, organized as rows and columns. Each column is a vector (or factor) with the same number of rows, columns can have different data types, and the object has class "`data.frame`".

**Excel** (`.xlsx`): A Microsoft Excel workbook file (`.xlsx`) that can store one or more worksheets, along with spreadsheet features like formulas and formatting. `.xlsx` uses the Office Open XML format (the "x" indicates an XML-based file without macros).

**File Path**: A text "address" that tells the computer (and R) where a file or folder is located. Paths can be absolute (the full location from the start of the file system) or relative (interpreted relative to your current working directory). In R, `getwd()` shows the current working directory that relative paths are based on, and `file.path()` helps build paths in an operating-system-safe way.

**Graphical User Interface** (GUI): A point-and-click interface that lets you interact with software using menus, buttons, and file browsers instead of typing commands. In RStudio, the Import Dataset wizard is a GUI tool that helps you select a file and then outputs the R code (including the file path) needed to reproduce the import, so you can copy that code into your R script.

**Package**: An installable collection of R code—functions, documentation, and sometimes datasets—that adds new features beyond base R. Packages are commonly installed from CRAN and then loaded with `library()` so you can use their tools in your session.

**Rows**: The horizontal records in a data frame or table. In tidy tabular data, each row usually represents one observation, case, or unit being measured, such as one person, car, state, experiment, or survey response.

**Tabular Data**: Data arranged in a table (a rectangular structure) where rows represent records/observations

and columns represent variables/attributes. This row–column layout is the standard format used by spreadsheets, databases, and data frames.

**Working Directory**: The folder on your computer that R treats as the default "home base" for the current session. When you read files (e.g., `read.csv("data.csv")`) or save outputs (e.g., `write.csv(df, "results.csv")`) using relative paths, R looks in (and writes to) the working directory unless you specify a full path. You can view or change it using Session → Set Working Directory, and you can check it with `getwd()` or change it with `setwd("path/to/folder")`.

# 7 Practice Space

In most chapters, the Guided Activity helps you create a notes file first, and then the Practice Space asks you to create a separate practice file.

Chapter 7 works differently because importing data is not only about typing the correct R function. Importing data requires understanding your folder structure and careful step-by-step verification and troubleshooting. For this reason, the Guided Activity helps you begin building the same `chapter7_practice.Rmd` file that you will use for this Practice Space.

After completing the Guided Activity, please tidy the formatting and add comments to your `chapter7_practice.Rmd` file. The practice space below gives you a checklist of the tasks to complete and prompts for writing comments and reflections.

By the end of this Practice Space, your Module_4 folder should contain:

```
Intro_to_R/
  Module_4/
      chapter7_practice.Rmd
      chapter7_practice.RData
      Values_Transport_dplace.csv
      car_prices.xlsx
```

Your R environment should contain:

- `mtcars`
- `movielens`
- `dplace_df`
- `car_prices_df`

In this Practice Space, you will see two kinds of memo prompts:

- Add Comments asks you to explain what the code or output shows.
- Reflect asks you to explain your thinking process, what confused you, what changed in your understanding, or what strategy you would use next.

These are there to help you practice writing memos as you work. In each task, you can choose whether to answer one, two, or all memo prompts.

**Autonomy** is the habit of taking ownership of your thinking, choices, code, and learning process while using help responsibly. Show autonomy by checking your own file locations, reading your output carefully, using error messages as evidence, and explaining what you would check next if something did not work.

If you encounter an error, document your **tenacity** by briefly noting what happened, what you tried, and how you fixed it or what still confuses you. Document how you troubleshot the error and what resource(s) you used to help you resolve the issue (**transparency**). Show **courage** by sharing your thinking process, even if it includes mistakes or confusion.

You may quote AI output only when you clearly mark it as quoted text, either with quotation marks or a block quote using `>`. Then hand-type your own explanation of why you are including that quote.

Before you finish, clear your Environment and run the notebook from top to bottom. Then knit to HTML to check that the notebook works in order.

## 7.1 Task 1

**Organize your files**

Download the two data files for this chapter and place them in the same folder as your R Notebook.

Files for this chapter:

- `Values_Transport_dplace.csv`
- `car_prices.xlsx`

Check your folder in Finder or File Explorer. You should be able to see your Rmd file and both data files together in the same folder.

Add Comments: Explain why keeping the Rmd file and the data files in the same folder makes importing data simpler in this chapter.

## 7.2 Task 2

**Check where R is looking**

Run `getwd()` and `list.files()` in the **Console**. Then include the same code in a code chunk of your R Notebook and test Knit.

### 7.2.1 Step 1

Type and Run this code in the Console first

```
getwd()
list.files()
```

### 7.2.2 Step 2

Add a code chunk in your R Notebook, write the same code in that code chunk, then knit your notebook.

```
#  Type this in your notebook
getwd()
list.files()
```

### 7.2.3 Step 3

Check the resulting HTML and compare with Console output.

Reflect: Did R look in the same folder both times, or different folders? What does this tell you about how R handles file paths when you run code in the Console vs. when you Knit an R Notebook? Why would this matter when you are importing data?

## 7.3 Task 3

**Load packages**

Load the packages `readr`, `readxl`, and `dslabs` into your R session.

```
_______(readr)
_______(readxl)
_______(dslabs)
```

Check that the packages are installed by running `packageVersion()`for each package.

```
_______(_______)
_______(_______)
_______(_______)
```

Add Comments: Explain the difference between installing a package one time and loading a package each time you start a new R session.

## 7.4 Task 4

**Load a Base R dataset**

Load the `mtcars` dataset from base R.

```
_______(_______)
```

Check that `mtcars` loaded correctly.

Add Comments: Explain why `mtcars` can be loaded with `data()` instead of imported from a file on your computer.

## 7.5 Task 5

**Load a curated package dataset**

Load the `movielens` dataset from the `dslabs` package.

```
_______(_______, package = "_______")
```

Check that `movielens` loaded correctly.

Add Comments: Explain what the `package = "dslabs"` part tells R to do.

## 7.6 Task 6

**Import a CSV file**

Import the CSV file `Values_Transport_dplace.csv` to R using `read_csv()` and store it as `dplace_df`.

```
dplace_df <- _______(_______)
```

Check that the data imported correctly.

Reflect: Explain how you knew the CSV import worked. Use evidence from the import summary or your checks, such as the number of rows and columns, the comma delimiter, or the column types. Then name one likely cause of an error if this import failed: the file was not in the same folder as the Rmd file, the file name was typed differently, or `library(readr)` had not been run.

## 7.7 Task 7

**Import an Excel file**

Import the Excel file `car_prices.xlsx` to R using `read_excel()` and store it as `car_prices_df`.

```
car_prices_df <- _______(_______)
```

Check that the data imported correctly.

Reflect: Explain how importing an Excel file is similar to and different from importing a CSV file. Name the package/function used for the Excel file, explain why the file can be imported with only `"car_prices.xlsx"` instead of a longer file path, and describe one thing you would check if the import failed or the data did not look right.

## 7.8 Task 8

**Save your workspace for submission**

Save your workspace (`chapter7_practice.RData`) for submission.

This file should contain:

- `car_prices_df` (the data frame created by importing the Excel file)
- `dplace_df` (the data frame created by importing the CSV file)
- `movielens` (the data frame created by loading the dataset from the `dslabs` package)
- `mtcars` (the data frame created by loading the dataset from base R)

```
save(
  car_prices_df,
  dplace_df,
  movielens,
  mtcars,
  file = "chapter7_practice.RData"
)
```

Optional: Show off your coding skills add `car_prices_sheet1` by editing the code above. Only do this if `car_prices_sheet1` is in your environment.

Check that the `.RData` file was created and is saved in your module folder.

## 7.9 Task 9

In this course, what is tabular data?

A. A list where each element can be a different length

B. A table where rows represent observations and columns represent variables

C. A single vector that stores only numbers

D. A plot with x- and y-axes

## 7.10 Task 10

```
list.files()
```

What does the output tell you?

A. Which packages are installed on your computer

B. Which functions are available in Base R

C. Which files are in R's current working directory

D. Which objects are stored in the Environment pane

## 7.11 Task 11

You click Knit and get an error that a file cannot be opened. What is the best first check?

A. Reinstall R and all packages

B. Run `getwd()` and `list.files()` to confirm where R is looking and whether the file is there

C. Prompt an LLM with the error message to get suggestions for how to fix it

D. Rename the file to something shorter

### 7.12 Task 12

Which function is used in this chapter to import an Excel `.xlsx` file?

A. `read_csv()`

B. `read_excel()`

C. `readxl()`

D. `readr()`

### 7.13 Task 13

Which function is used in this chapter to import a CSV file?

A. `read_csv()`

B. `read_excel()`

C. `readxl()`

D. `readr()`